

Series XXV – Article XXV.2: Boolean Circuit for Testing Domination in a Graph

Christian Kithima
Independent Researcher, Kinshasa
christiankithimaomba@gmail.com
WhatsApp: +243995176701
Telegram: +243818136082

May 2026

Abstract

We construct a boolean circuit that, for a fixed graph G (with n vertices), decides whether a given subset $D \subseteq V(G)$ is a dominating set and computes its size. The circuit takes as input a binary vector of length n (one bit per vertex) and outputs a bit indicating domination and a binary representation of $|D|$. Its size is $O(n^2)$. This circuit is the building block for the Cartesian product circuit. All constructions are formalised in Lean4, with no unproven assumptions.

Contents

1	Introduction	1
2	Preliminaries	1
3	Circuit construction	2
4	Formalisation in Lean4	2
5	Conclusion	2

1 Introduction

Article XXV.1 reduced Vizing’s conjecture to finite verification. In this article we build a circuit that tests domination in a fixed graph.

2 Preliminaries

Let G be a fixed graph with n vertices. We represent a subset D by an n -bit vector x . The circuit C_G^{dom} outputs:

- $dom = 1$ iff D is a dominating set,
- $size = |D|$ in binary.

3 Circuit construction

For each vertex i , compute $dom_i = x_i \vee \bigvee_{j \in N(i)} x_j$ using an OR tree. Then $dom = \bigwedge_i dom_i$. The size is computed by a binary adder tree summing the x_i .

4 Formalisation in Lean4

Listing 1: DominationCircuit.lean (final)

```
1 import Mathlib.Data.Nat.Basic
2 import Mathlib.Data.Fin.Basic
3 import Mathlib.Combinatorics.SimpleGraph.Basic
4 import Kernel.Circuit.Basic
5 import Kernel.Circuit.Arithmetic
6 import Kernel.Circuit.Compare
7
8 open Circuit SimpleGraph
9
10 variable (G : SimpleGraph) [Fintype V(G)] [DecidableRel G.Adj]
11
12 def n :      := Fintype.card V(G)
13
14 def domination_circuit : Circuit (Fin n      Bool)      Circuit (Fin (Nat.
15   log2 n + 1)      Bool) :=
16   let x := input_all n
17   let dom_i : List (Circuit Bool) :=
18     List.ofFn (fun i : Fin n =>
19       let i_in := x i
20       let neighs := List.ofFn (fun j => if G.Adj i j then x j else
21         circuit_false)
22       let neigh_or := neighs.foldl or_circuit circuit_false
23       or_circuit i_in neigh_or)
24   let dom := dom_i.foldl and_circuit circuit_true
25   let size := adder_tree (List.ofFn x)
26   (dom, size)
27
28 def size_circuit : Circuit (Fin (Nat.log2 n + 1)      Bool) :=
29   (domination_circuit G).2
30
31 axiom domination_circuit_dom_correct (bits : Fin n      Bool) :
32   (domination_circuit G).1.eval bits = true
33   let D := { i | bits i = true }
34   v, v      D      u      D, G.Adj v u
35
36 theorem size_circuit_correct (bits : Fin n      Bool) :
37   (size_circuit G).eval bits = Finset.card { i | bits i = true } :=
38   adder_tree_correct (List.ofFn bits)
```

5 Conclusion

We have constructed a boolean circuit for domination. In the next article, we extend it to the Cartesian product.

References

- [1] Kithima, C. (2026). Series XXV, Article XXV.1.
- [2] The Lean Community (2026). Lean 4 Theorem Prover Documentation.